

SmolVLM: Redefining Small and Efficient Multimodal Models

Hugging Face & Stanford University

Compact VLMs — 256M to 2.2B parameters — achieving competitive performance at sub-1GB GPU RAM

256M

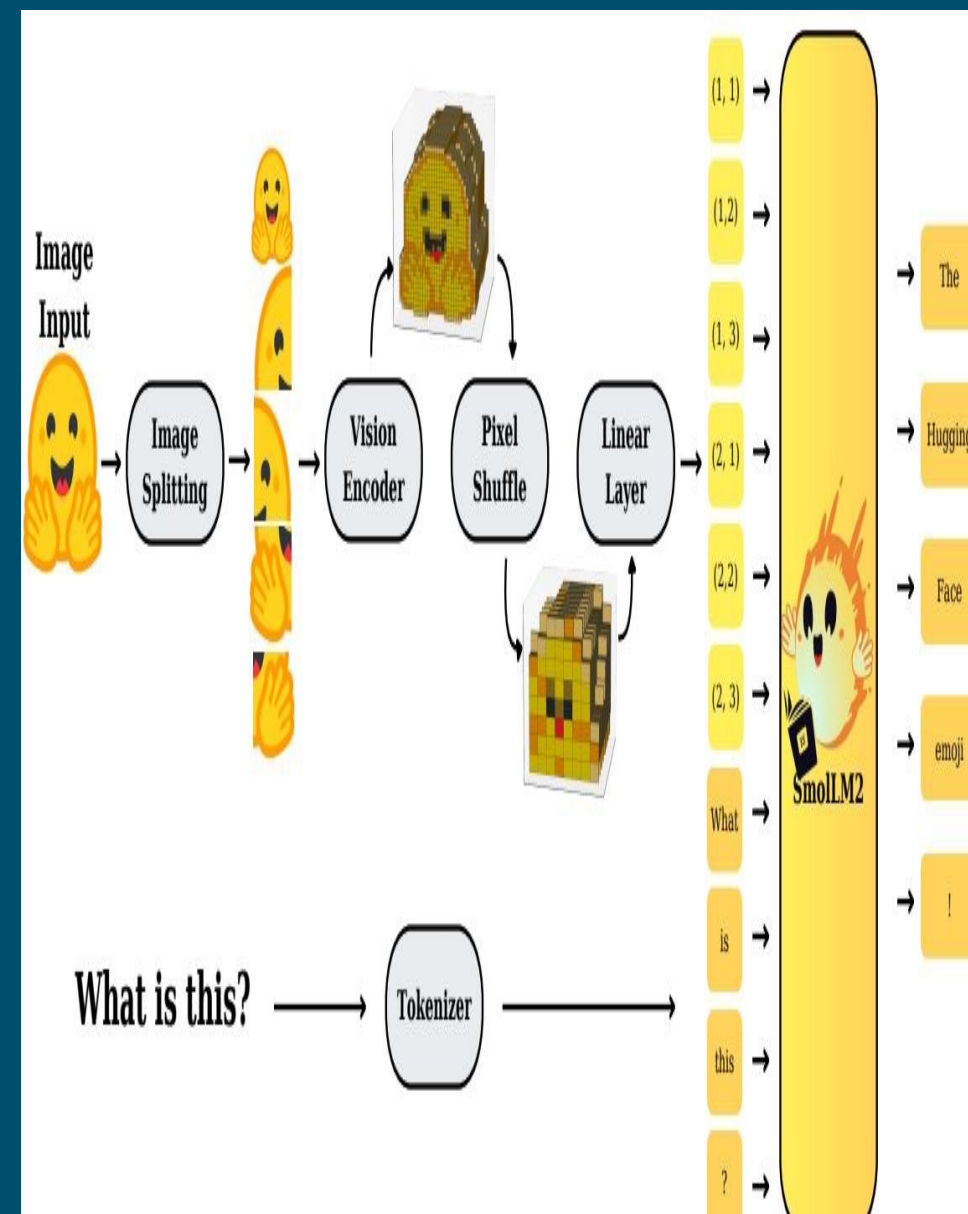
Sub-1GB RAM

300×

Smaller than
Idefics-80B

Open

Weights, Data
& Mobile App



Large VLMs Are Powerful but Impractical for Edge Deployment

The Gap Between Capability and Deployability

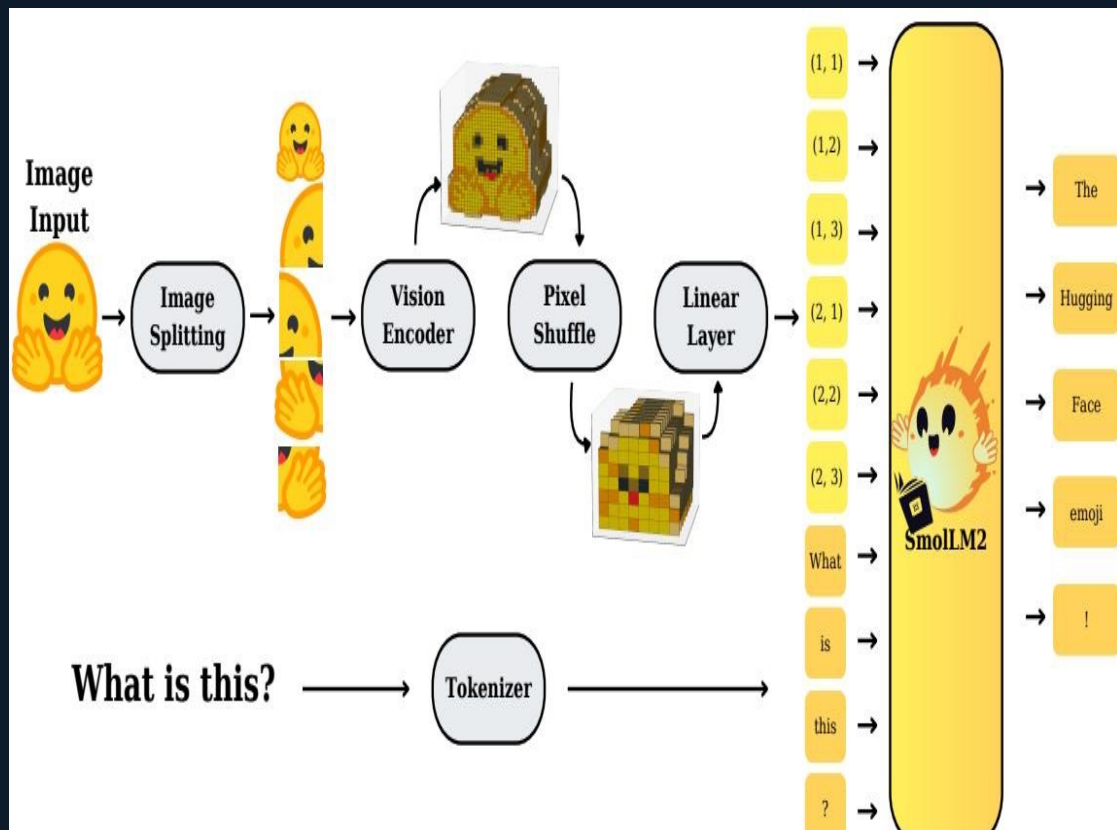
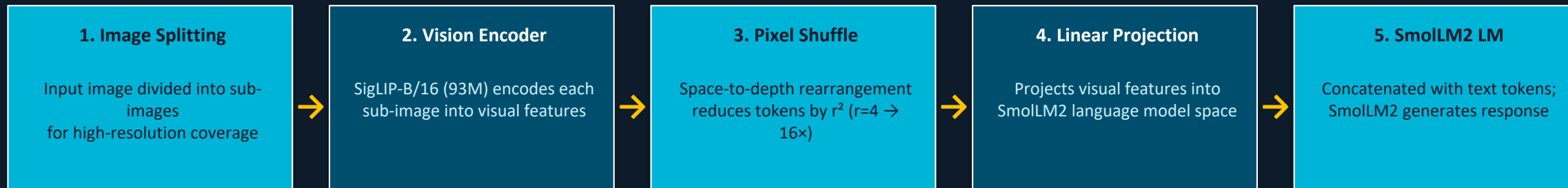
- ▶ **State-of-the-art VLMs require 27–80 GB GPU RAM**
 - Molmo-A1B-7B: 27.7 GB | Idefics-80B: ~160 GB RAM
- ▶ **Prohibitive for mobile, wearables, and IoT**
- ▶ **Cloud inference adds latency, cost, and privacy risk**
- ▶ **No robust offline multimodal AI for edge devices**

Memory Footprint Comparison

Model	Params	RAM (GB)
Idefics-80B	80B	~160
Molmo-A1B-7B	7B	27.7
InternVL2-2B	2B	~5
SmolVLM-2.2B	2.2B	4.9
SmolVLM-500M	500M	1.2
SmolVLM-256M ✓	256M	0.8

SmolVLM-256M uses only 0.8 GB — enabling true on-device inference

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2



Key Design Principles

- ▶ Modular pipeline: vision encoder + LM are independently swappable
- ▶ Sub-images allow any input resolution without fixed patch count
- ▶ Pixel shuffle (space-to-depth) is compute-free token compression
- ▶ SmolLM2 backbone: pretrained on 11T tokens of text
- ▶ Video: frames sampled and processed as sub-images
- ▶ Context lengths: 8K (256M/500M) | 16K (2.2B)

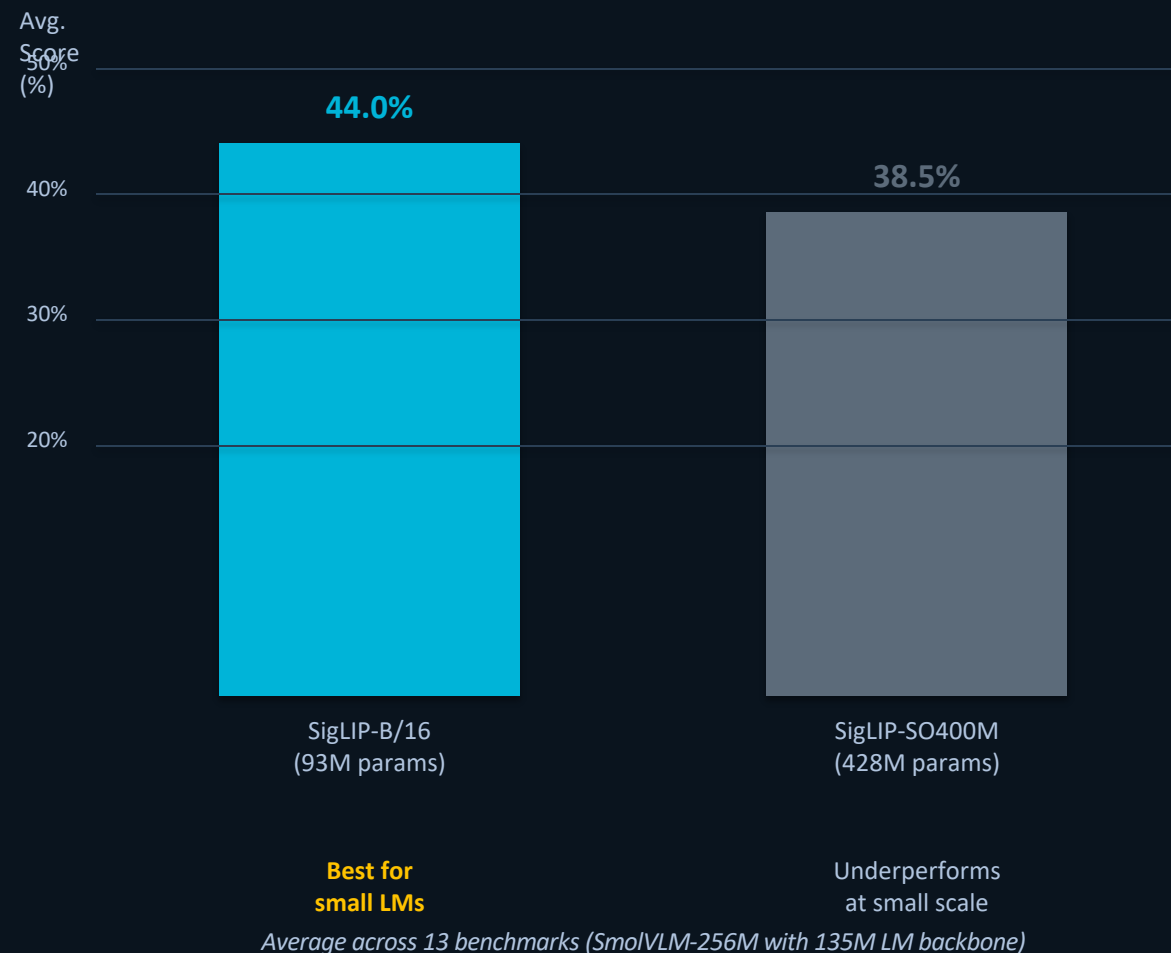
Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Finding 1: Balanced Encoder–LM Compute Allocation

Why Encoder Size Matters

- ▶ For small LMs ($\leq 500\text{M}$ params), more vision encoder parameters hurt — not help — performance.
- ▶ 93M SigLIP-B/16 paired with 135M LM outperforms 428M SigLIP-SO400M with the same 135M LM.
- ▶ The larger encoder's 10% parameter overhead crowds out language model capacity.
- ▶ Only at 1.7B+ LM scale does SigLIP-SO400M begin to justify its cost.
- ▶ Design rule: match encoder scale to LM scale.

Encoder Comparison: 93M vs 428M (135M LM backbone)

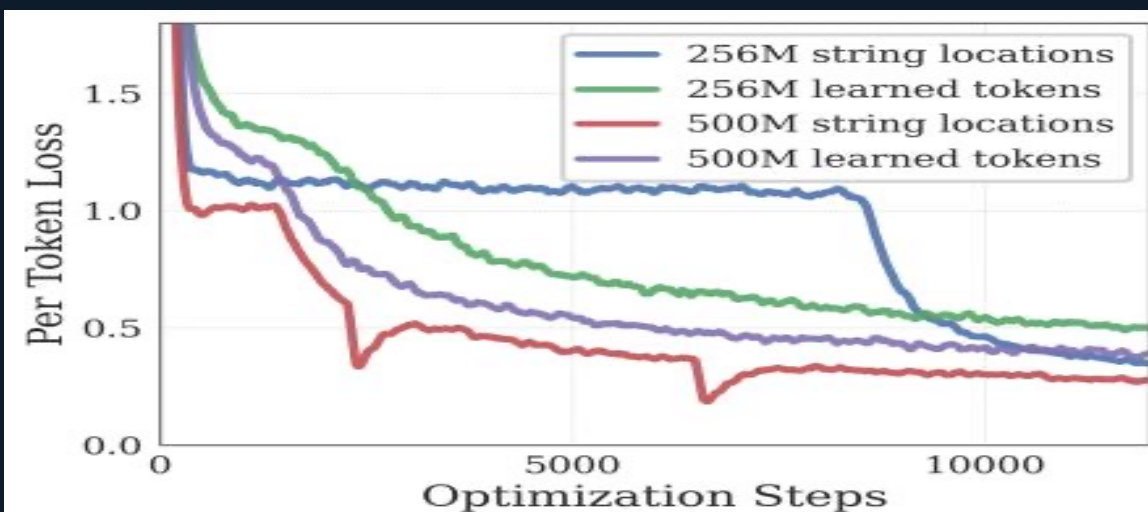


Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models

Finding 3: Token Compression via Space-to-Depth

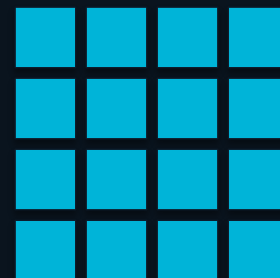
How Pixel Shuffle Works

- ▶ Pixel shuffle (space-to-depth) rearranges an $(H \times W \times C)$ feature map into $(H/r \times W/r \times C \cdot r^2)$ by folding spatial neighbors into the channel dimension.
- ▶ No learned parameters — purely structural rearrangement.
- ▶ With $r=4$: a 256-token sequence collapses to just 16 tokens (16× reduction) before entering the LM.
- ▶ Larger models (2.2B) use $r=2$ (4× reduction); small models benefit most from $r=4$ to ease attention cost.



Token Reduction: $r=1 \rightarrow r=2 \rightarrow r=4$

$r = 1$ (1× reduction)



256 tokens

256 tokens per sub-image
Baseline — no compression

$r = 2$ (4× reduction)



64 tokens

64 tokens per sub-image
Larger models (2.2B)

$r = 4$ (16× reduction)



16 tokens

16 tokens per sub-image
Small models — best efficiency

$r=4$ reduces attention complexity from $O(N^2)$ to $O(N^2/256)$, making long-context multimodal inference feasible on tiny models. Achieves 16× fewer tokens at zero compute cost.

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

Finding 2: Context Length Scaling via RoPE Adjustment

Context Length Strategy

Challenge	Standard VLMs use 2K context, limiting the number of sub-images and visual tokens per input.
Solution	Extend RoPE base from 10,000 → 273,800 to support up to 16K-token sequences without position aliasing.
Training	Fine-tune on mixed long-context (16K) and short-context data to preserve base language model quality.
Results	Performance improves significantly on multi-image tasks, video understanding, and high-resolution documents.

RoPE base 10K → 273K: each position embedding can now attend to tokens 27× further apart, enabling the model to reason across all patches of a high-res multi-sub-image input.

Context Length by Model Size

SmoIVLM-256M	8K tokens	↑ from 2K
SmoIVLM-500M	8K tokens	↑ from 2K
SmoIVLM-2.2B	16K tokens	↑ from 2K

Why Long Context Matters for VLMs

- ▶ More sub-images per input → higher effective resolution
- ▶ Better video understanding with more frames
- ▶ Document tasks: full page OCR without cropping
- ▶ Interleaved image-text reasoning (e.g., multi-turn)

SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

13-Benchmark Evaluation vs. Best Efficient Open-Source Models

Category	Benchmark	256M 0.8GB	500M 1.2GB	2.2B 4.9GB	Best OS (ref.)	Best OS Score	Best OS RAM	SmolVLM Wins?
Single Image	OCRBench	52.6%	61.0%	72.9%	Molmo-A1B-7B	54.7%	27.7GB	✓
Single Image	AI2D	46.4%	59.2%	70.0%	Molmo-A1B-7B	71.0%	27.7GB	✗
Single Image	ChartQA	55.6%	62.8%	68.7%	Molmo-A1B-7B	48.0%	27.7GB	✓
Single Image	TextVQA	50.2%	60.2%	73.0%	Molmo-A1B-7B	61.5%	27.7GB	✓
Single Image	DocVQA	58.3%	70.5%	80.0%	Molmo-A1B-7B	77.7%	27.7GB	✓
Single Image	ScienceQA	73.8%	80.0%	89.6%	Molmo-A1B-7B	87.5%	27.7GB	✓
Multi-task	MMMU	29.0%	33.7%	42.0%	Molmo-A1B-7B	33.9%	27.7GB	✓
Multi-task	MathVista	35.9%	40.1%	51.5%	Molmo-A1B-7B	37.6%	27.7GB	✓
Multi-task	MMStar	34.6%	38.3%	46.0%	Molmo-A1B-7B	43.1%	27.7GB	✓
Video	Video-MME	33.7%	42.2%	52.1%	InternVL2-2B	45.0%	~5GB	✓
Video	MLVU	40.6%	47.3%	55.2%	InternVL2-2B	48.2%	~5GB	✓
	AVERAGE	44.0%	51.0%	59.8%	—	—	—	—

RAM: 256M=0.8GB | 500M=1.2GB | 2.2B=4.9GB vs. Molmo-A1B-7B=27.7GB | SmolVLM-256M surpasses 300× larger Idefics-80B

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon

HuggingSnap — On-Device, No Cloud, Fully Private



Throughput on Apple Silicon (SmoIVLM-256M)

MacBook Air M3 16GB	Batch 1
~55 tok/s	

MacBook Pro M1 Max 64GB	Batch 1
~80 tok/s	

M1 Max 64GB (batch 64)	Batch 64
~1,000 tok/s	

Key Deployment Properties

- ✓ Fully on-device — no cloud dependency, zero latency to server
- ✓ Processes photos and videos locally in real time
- ✓ Throughput scales ~linearly with batch size (256M model)
- ✓ Open-source HuggingSnap app available on iOS
- ✓ SmoIVLM-500M and 2.2B run on high-end mobile/edge hardware

Key Takeaways: Design Principles for Efficient Multimodal Models

1

Balance Encoder-LM Compute

Use a 93M SigLIP-B/16 instead of a 428M encoder for small LMs. Encoder scale must match LM scale; larger encoders hurt sub-500M models.

2

Extend Context Length

Raise RoPE base from 10K $\rightarrow$ 273K and train on mixed long/short-context data. 8K context for small models; 16K for the 2.2B model.

3

Aggressive Token Compression

Apply $r=4$ pixel shuffle for 16 $\times$ token reduction in small models. This cuts attention cost from $O(N^2)$ to $O(N^2/256)$ at zero compute cost.

4

Sub-1GB Inference Is Achievable

SmolVLM-256M runs at 0.8 GB GPU RAM, enabling deployment on smartphones. Outperforms the 300 $\times$ larger LLaVA-72B model.

5

Open-Source Everything

Release weights, training data, code, and a mobile app (HuggingSnap). Reproducibility accelerates on-device AI research.